

PAW — INTEGRACIÓN

Frontend + Spring

Por qué cada paso existe — no solo cómo hacerlo

Maven module

frontend-maven-plugin

SPA routing

base path

CONTEXTO

En desarrollo: dos servidores, dos mundos

Frontend

```
npm start / vite dev
```

```
→ http://localhost:3000 o 5173
```

Hot reload, errores en el browser, DX óptima.

Backend

```
mvn tomcat7:run
```

```
→ http://localhost:8080
```

API REST, base de datos, lógica de negocio.

El frontend llama a la API en otro puerto — esto funciona en desarrollo con un proxy o configurando CORS:

```
// vite.config.js — proxy en dev
server: {
  proxy: {
    '/api': 'http://localhost:8080'
  }
}
```

En desarrollo esto alcanza. Cada proceso corre independiente, el proxy redirige las llamadas a la API. ¿Pero qué pasa cuando hay que poner la app en producción?

CONTEXTO

En producción: un solo servidor para todo

× Dev: dos procesos separados

`frontend` `localhost:3000`

`mi-app` `localhost:8080`

En un servidor de producción no podés correr `npm start` y esperar que siga funcionando. El frontend tiene que ser archivos estáticos que alguien sirva.

✓ Prod: Tomcat sirve todo

`Tomcat` `servidor:8080/mi-app`

`GET /mi-app/` → `index.html` (React/Angular)
`GET /mi-app/static/...` → JS, CSS, assets
`GET /mi-app/api/pets` → Spring Controller

El WAR de Spring incluye el build del frontend. Tomcat sirve los archivos estáticos **y** las requests a la API desde el mismo proceso.

El objetivo de esta guía: lograr que `mvn package` genere ese WAR unificado automáticamente.

EL PROBLEMA

Dos mundos, dos builds — sin coordinación

Para deployar la app, hay que recordar correr dos builds en orden correcto:

cd frontend

→

npm install

→

npm run build

→

mvn clean package

Cada paso es manual. Si alguien olvida el build del frontend antes del mvn package, el WAR se genera con el JS de la versión anterior — o sin JS en absoluto.

¿Qué pasa cuando alguien del equipo olvida el build del frontend antes del deploy?

SOLUCIÓN — PASO 1

El frontend como **módulo Maven**

✗ Proceso manual — frágil

```
cd frontend && npm install
```

```
npm run build
```

```
mvn clean package
```

Tres comandos en orden. Sin automatización, sin garantías. Un olvido y el deploy es incorrecto.

✓ Frontend como módulo — un solo comando

```
my-project/  
├─ frontend/           ← nuevo módulo Maven  
│  ├─ pom.xml  
│  └─ src/             ← código React/Angular/Vite  
├─ webapp/  
│  └─ pom.xml  
├─ models/  
│  └─ pom.xml  
└─ pom.xml             ← padre
```

Un solo **mvn package** desde el root buildea el frontend **y** el backend. Maven orquesta el orden automáticamente.

EL PROBLEMA

Maven habla Java — no sabe qué es npm

Si agregás el módulo frontend/ con un pom.xml vacío y corrés mvn package desde el root:

```
[INFO] Building frontend 1.0-SNAPSHOT
[INFO] --- maven-resources-plugin ... ---
[INFO] --- maven-compiler-plugin ... ---
[INFO] BUILD SUCCESS
```

← Maven lo procesa como un módulo Java vacío.
Node no se instala. npm no corre. dist/ nunca se genera.

Maven no tiene conocimiento nativo de Node.js ni npm. El módulo compila exitosamente — pero el JS nunca se buildea.

¿Cómo le decís a Maven que corra npm?

El frontend-maven-plugin

✗ Sin el plugin

El módulo frontend/ existe en el árbol de Maven, pero:

- ✗ Node.js no se instala
- ✗ npm install no corre
- ✗ dist/ nunca se genera
- ✗ El WAR no tiene JS

¿Por qué instalar Node en el plugin?

Node y npm no están garantizados en los servidores de CI. El plugin descarga la versión exacta especificada, garantizando reproducibilidad.

✓ Con el plugin — 3 executions

frontend/pom.xml

```
<plugin>
  <groupId>com.github.eirslett</groupId>
  <artifactId>frontend-maven-plugin</artifactId>
  <executions>

    <execution>
      <id>install-node-and-npm</id>
      <goals><goal>install-node-and-npm</goal></goals>
      <configuration>
        <nodeVersion>v20.11.0</nodeVersion>
      </configuration>
    </execution>

    <execution>
      <id>npm-install</id>
      <goals><goal>npm</goal></goals>
    </execution>

    <execution>
      <id>npm-build</id>
      <goals><goal>npm</goal></goals>
      <configuration>
        <arguments>run build</arguments>
      </configuration>
    </execution>

  </executions>
</plugin>
```

EL PROBLEMA

El módulo existe — Maven no lo conoce

El directorio `frontend/` y su `pom.xml` ya existen. Pero el pom padre no los declara:

`pom.xml` (padre)

```
<modules>
  <module>models</module>
  <module>services</module>
  <module>webapp</module>
  ← frontend no está declarado
</modules>
```

Al correr `mvn package` desde el root, Maven ignora completamente el módulo `frontend/`. El plugin no corre, el JS no se buildea.

¿Qué pasa si corrés `mvn package` desde el root sin declarar el módulo?

Declarar en el **pom padre**

✗ Sin declaración — módulo ignorado

pom.xml (padre)

```
<modules>
  <module>models</module>
  <module>services</module>
  <module>webapp</module>
  ← frontend ausente
</modules>
```

✓ Con declaración + pluginManagement

pom.xml (padre)

```
<modules>
  <module>models</module>
  <module>services</module>
  <module>frontend</module>
  <module>webapp</module>
</modules>

<build>
  <pluginManagement>
    <plugins>
      <plugin>
        <groupId>com.github.eirslett</groupId>
        <artifactId>frontend-maven-plugin</artifactId>
        <version>1.15.0</version>
      </plugin>
    </plugins>
  </pluginManagement>
</build>
```

pluginManagement centraliza la versión del plugin en el padre.
El módulo hijo referencia el plugin sin repetir la versión — un solo lugar para actualizarla.

EL PROBLEMA

El dist/ existe — el WAR no lo sabe

Después de `mvn package`, el build del frontend existe en disco:

```
Angular → frontend/dist/paw-catedra/browser/  
React   → frontend/build/  
Vite     → frontend/dist/
```

```
├─ index.html  
├─ main.abc123.js  
└─ styles.def456.css
```

← Cada bundler elige su propia carpeta de salida.
Ninguno se incluye en el WAR automáticamente.

El WAR generado solo contiene clases Java y recursos del módulo `webapp/`. Navegás a `/` — no hay `index.html` del frontend.

¿Cómo se incluyen esos archivos en el empaquetado?

Incluir el dist/ con **webResources**

✗ WAR sin config extra

```
my-app.war
├─ WEB-INF/
│   ├── classes/
│   └─ lib/
└─ ← sin index.html
    sin main.js
    sin styles.css
```

Maven empaqueta solo lo que está en `webapp/src/main/webapp/`. El `dist/` del frontend vive en otro módulo — Maven no lo incluye solo.

✓ Con **webResources**

webapp/pom.xml

```
<plugin>
  <artifactId>maven-war-plugin</artifactId>
  <configuration>
    <webResources>
      <resource>
        <directory>
          ../frontend/dist/paw-catedra/browser
        </directory>
      </resource>
    </webResources>
  </configuration>
</plugin>
```

Cada bundler genera su propia carpeta. El path en

`<directory>` depende del framework:

Angular → `../frontend/dist/<app-name>/browser`

React → `../frontend/build`

Vite → `../frontend/dist`

Siempre verificá dónde queda el `index.html` después del build — ese es el directorio correcto.

Dependencia en webapp/pom.xml: agregar frontend como dependencia `scope=provided`, `type=pom` garantiza que Maven buildee el módulo frontend antes que webapp.

EL PROBLEMA

El servidor recibe rutas del SPA — no sabe qué hacer con ellas

Con el DispatcherServlet en `/*`, Spring intercepta **todas** las rutas. El resultado:

GET `/dashboard` → Spring no tiene controller → 404

GET `/api/pets/999` → Spring → recurso no existe → 404

GET `/static/main.js` → Spring no sirve archivos → 404

Si mapeamos *todo* 404 a `index.html` para resolver el routing del SPA, los errores reales de la API (`/api/pets/999` no existe) también redirigen a HTML. El cliente recibe una página web donde esperaba JSON.

Necesitamos que cada tipo de request vaya al lugar correcto — sin trucos de 404.

Routing declarativo en **web.xml** — sin código Java

web.xml

```
<!-- 1. Spring maneja solo la API -->
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>/api/*</url-pattern>
</servlet-mapping>

<!-- 2. Archivos estáticos → servlet por defecto -->
<servlet-mapping>
  <servlet-name>default</servlet-name>
  <url-pattern>/assets/*</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>default</servlet-name>
  <url-pattern>/static/*</url-pattern>
</servlet-mapping>

<!-- 3. Todo lo demás → index.html -->
<servlet>
  <servlet-name>index</servlet-name>
  <jsp-file>/index.html</jsp-file>
</servlet>
<servlet-mapping>
  <servlet-name>index</servlet-name>
  <url-pattern>/*</url-pattern>
</servlet-mapping>
```

¿Cómo resuelve Tomcat cada ruta?

GET /api/pets → coincide con `/api/*` → DispatcherServlet → Spring controller

GET /assets/main.js → coincide con `/assets/*` → servlet default → sirve el archivo

GET /dashboard → cae en `/*` → servlet index → sirve `index.html` → el router JS toma el control

`<jsp-file>` le dice a Tomcat que ese servlet sirve ese archivo HTML. Sin código Java — todo declarativo en `web.xml`.

EL PROBLEMA

La app está en /mi-app/ — los assets no lo saben

La app se deploya en Tomcat bajo un contexto: `http://localhost:8080/my-app/`

Por defecto, **todos los bundlers** asumen que el app vive en la raíz del servidor y generan paths absolutos:

```
<link href="/static/css/main.css"> ← React, Vite, Angular sin config  
<script src="/static/js/main.js"></script>
```

El browser pide GET `/static/css/main.css` → 404
Debería pedir GET `/my-app/static/css/main.css`

Cada bundler tiene su propia clave para configurar esto: `homepage`, `base`, `baseHref`... el nombre varía pero el concepto es el mismo.

¿Cómo le decís a tu bundler que los assets están bajo un subdirectorio?

Configurar el **base path** del bundler

✗ HTML generado sin base path

```
<link href="/static/css/main.css">
```

↑ path absoluto desde raíz
el browser pide /static/css/main.css
→ 404 (la app está en /my-app/)

Sin configuración, React y Vite asumen que la app vive en la raíz del servidor (/). Si está bajo un contexto (/my-app/), todos los assets fallan.

✓ Configurar el bundler

React — package.json

```
{  
  "homepage": "/my-relative-path"  
}
```

Vite — vite.config.js

```
export default defineConfig({  
  base: "/my-relative-path",  
});
```

Angular — angular.json

```
"build": {  
  "options": {  
    "baseHref": "/my-relative-path/"  
  }  
}
```

Resultado: `<link href="/my-relative-path/static/css/main.css">` ✓

Otro bundler: cada herramienta tiene su propia clave de configuración. Busca en la documentación oficial cómo establecer el *base path* o *public URL* antes de hacer el build.

RESUMEN

El camino completo — por qué existe cada paso



Qué resolvimos

- ✓ Build unificado con un solo mvn package
- ✓ Node.js reproducible en cualquier entorno (incluido CI)
- ✓ Assets del frontend incluidos en el WAR
- ✓ DispatcherServlet en /api/* — Spring solo maneja la API
- ✓ Servlet index (<jsp-file>) en /* — rutas SPA → index.html sin código Java
- ✓ default servlet en /assets/*, /static/* — archivos estáticos servidos directamente
- ✓ Assets con paths correctos bajo el contexto del servidor

Qué no hay que olvidar

- ✓ Agregar frontend como <module> en el pom padre
- ✓ Declarar frontend como dependencia en webapp/pom.xml (garantiza orden de build)
- ✓ El path en <directory> depende del bundler y del nombre del proyecto
- ✓ Configurar base / homepage con la ruta real del contexto Tomcat
- ✓ La ruta de base path en el bundler debe coincidir con el context-path del WAR